

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине: «Модели решения задач в интеллектуальных системах»

Тема: «Многослойный персептрон для задачи «исключающее ИЛИ» с числовыми классами: сравнение функций потерь MSE и Binary Cross-Entropy»

Выполнил:

Студент 4 курса

Группы ИИ-25

Трошин И.Д.

Проверила:

Андренко К.В.

Цель работы: На основе многослойного персептрона архитектуры 2-2-1, реализованного в Лабораторной работе №1 для задачи XOR с числовыми классами c_0 и c_1 из диапазона $[-10; 10]$, реализовать альтернативную функцию потерь – бинарную кросс-энтропию (Binary Cross-Entropy, BCE) – и провести сравнительный анализ обучения сети с MSE (Лабораторная работа №1) и с BCE по скорости сходимости, качеству обучения и устойчивости к выбору начальных весов.

Задачи лабораторной работы

1. Взять за основу код многослойного персептрона 2-2-1 из Лабораторной работы №1.

2. Поскольку BCE определена для выходов в диапазоне $(0; 1)$, а целевые значения в вашем варианте – произвольные числа c_0 и c_1 из диапазона $[-10; 10]$, реализовать нормализацию (масштабирование) целевых значений перед обучением и обратное преобразование выхода сети после обучения:

- привести c_0 и c_1 к бинарным меткам 0 и 1;
- обучать сеть на нормализованных целевых значениях с функцией потерь BCE;
- при выводе результата преобразовывать выход сети $\hat{y} \in (0; 1)$ обратно в шкалу $[c_0; c_1]$.

3. Реализовать корректный градиент для связки BCE + сигмоида на выходном слое; для скрытого слоя градиент передаётся по стандартному правилу цепочки.

4. Обучить сеть на полной таблице истинности своего варианта (4 примера) в двух конфигурациях:

- **Конфигурация А (базовая, из ЛР №1):** MLP 2-2-1, функция потерь MSE, целевые значения – исходные c_0/c_1 (без нормализации), выходной слой – сигмоида. Результаты можно взять из отчёта по ЛР №1 или, при необходимости, повторить обучение с фиксацией всех метрик.

- **Конфигурация Б (новая):** MLP 2-2-1, функция потерь BCE, целевые значения – нормализованные $(0/1)$, с последующим пересчётом выхода в шкалу $[c_0; c_1]$.

- Критерий остановки для обеих конфигураций одинаков – суммарная ошибка $E_s \leq E_e$ (используйте то же значение E_e , что и в ЛР №1, либо согласуйте новое значение отдельно для MSE и для BCE, поскольку это разные по своей природе величины – обоснуйте выбор в отчёте).

5. Для каждой конфигурации зафиксировать:

- количество эпох до достижения критерия остановки (либо факт недостижения за максимально допустимое число эпох);

- итоговую суммарную ошибку;
- ассигасу на всех 4 примерах таблицы истинности;
- среднюю абсолютную ошибку в исходной шкале $[c_0; c_1]$.

6. Провести не менее 5 запусков обучения для каждой конфигурации с разной случайной инициализацией весов (менять seed) и сравнить устойчивость сходимости: в скольких из 5 запусков сеть достигла критерия остановки, и какой разброс по числу эпох наблюдается.

7. Построить общий график зависимости суммарной ошибки от номера эпохи для конфигурации А (MSE) и конфигурации Б (BCE) на одних осях (для представительного запуска каждой конфигурации).

8. Реализовать блок визуализации результатов, включающий:

- **График сходимости** – суммарная ошибка E_s от номера эпохи, конфигурации А и Б на одних осях, разными цветами/стилями линий, с обозначением порога E_e .

- **Диаграмму разброса числа эпох по 5 запускам** (bar chart или scatter) для конфигурации А и конфигурации Б – наглядно показывающую устойчивость сходимости из п. 6 (например, столбики числа эпох для каждого из 5 seed, сгруппированные по конфигурации; запуски, не достигшие критерия остановки, отметить отдельно, например штриховкой или отдельным цветом).

- **Визуализацию разделяющей поверхности скрытого слоя** – для представительного обученного экземпляра каждой конфигурации построить на плоскости (А, В) карту выходного значения сети (heatmap или contour plot) с наложенными точками обучающей выборки (4 примера таблицы истинности), закрасенными по истинному классу. Диапазон осей А и В – $[-10; 10]$ (весь допустимый диапазон входных значений). Для конфигурации Б цветовую шкалу можно строить как по нормализованному $\hat{y} \in (0; 1)$, так и по пересчитанному значению в шкале $[c_0; c_1]$ – выбрать более наглядный вариант и обосновать выбор.

- **Диаграмму сравнения точности восстановления шкалы** – столбчатую диаграмму средней абсолютной ошибки для конфигурации А и конфигурации Б, чтобы наглядно сопоставить точность регрессии к исходной шкале $[c_0; c_1]$.

- Каждый график должен иметь подписанные оси, заголовок и легенду (где применимо). Разделяющую поверхность и графики точности рекомендуется разместить рядом (например, subplot 2×2 или отдельными рисунками) для удобства сравнения в отчёте.

9. Реализовать режим функционирования сети: пользователь вводит пару чисел (А, В) из диапазона $[-10; 10]$ (не обязательно совпадающих с c_0 и c_1). Сеть должна выводить:

- выход в нормализованной шкале $\hat{y} \in (0; 1)$ (с точностью до 4 знаков);

- выход, пересчитанный в исходную шкалу $[c_0; c_1]$ по формуле из п. 2;
- к какому классу (c_0 или c_1) ближе полученное значение.

Продemonстрировать работу режима функционирования на всех 4 примерах исходной таблицы истинности и не менее чем на 2–3 дополнительных парах (A, B), не входящих в обучающую выборку.

Вариант 13

Вариант 13 ($c_0 = 4, c_1 = -1$)

A	B	$A \oplus B$
4	4	4
4	-1	-1
-1	4	-1
-1	-1	4

Ход работы

Описание изменений в архитектуре и алгоритме обучения по сравнению с ЛР №1: формула нормализации целевых значений, формула обратного пересчёта выхода, вывод градиента для BCE + сигмоида.

Нормализация:

```
def normalize(value, c0, c1):
    return (value - c0) / (c1 - c0)

def denormalize(value, c0, c1):
    return c0 + value * (c1 - c0)
```

Обучение с BCE:

```
def online_train_bce(X, e, alpha=0.9, Ee=0.01, max_epochs=10000, seed=42):
    np.random.seed(seed)
    W1 = np.random.uniform(-0.1, 0.1, (2, 2))
    T1 = np.random.uniform(-0.1, 0.1, 2)
    W2 = np.random.uniform(-0.1, 0.1, 2)
    T2 = np.random.uniform(-0.1, 0.1)
    errors = []
    epoch = 0

    while True:
        for xi, ei in zip(X, e):
            S1, h, S2, y = forward(xi, W1, T1, W2, T2)

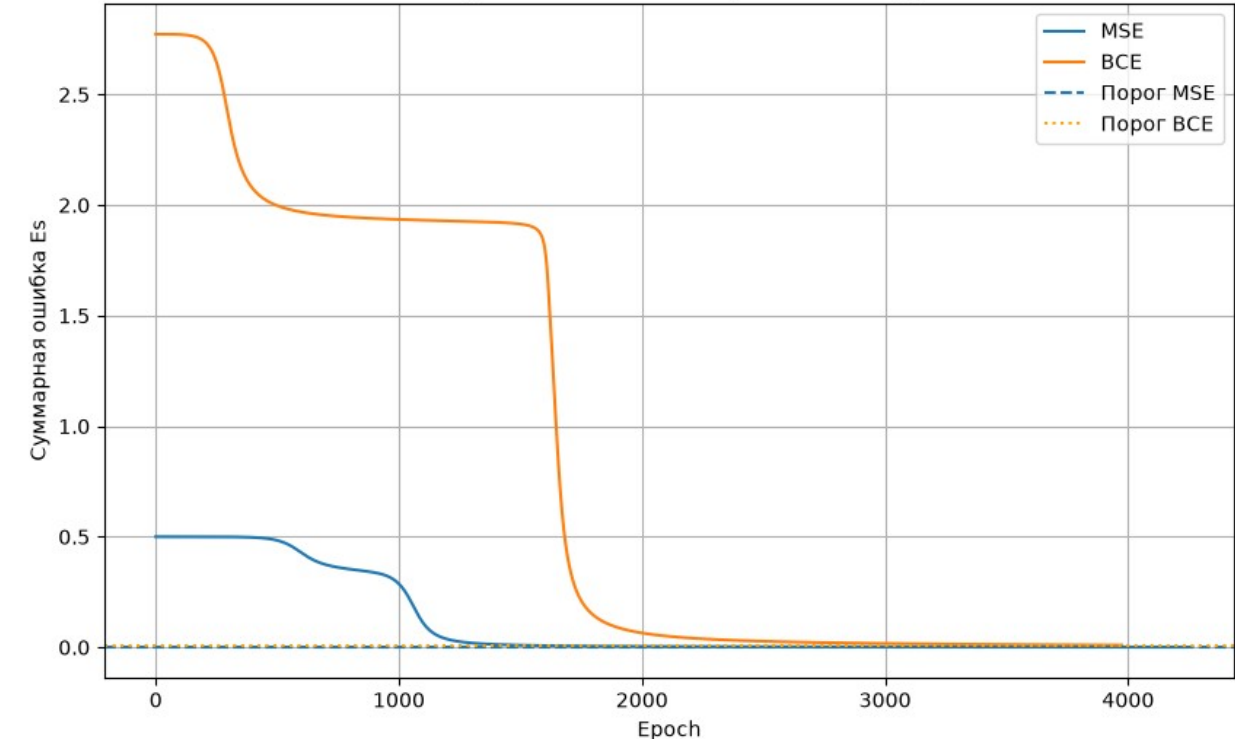
            delta2 = y - ei
            delta1 = (activation_derivative(S1) * W2 * delta2)

            W2 = W2 - alpha * h * delta2
            T2 = T2 + alpha * delta2
            W1 = W1 - alpha * np.outer(xi, delta1)
            T1 = T1 + alpha * delta1
        Es = 0
        for xi, ei in zip(X, e):
            y = predict(xi, W1, T1, W2, T2)
            y = np.clip(y, 1e-10, 1 - 1e-10) # защита от log(0)
            Es += -(ei * np.log(y) + (1 - ei) * np.log(1 - y))
        errors.append(Es)
        epoch += 1

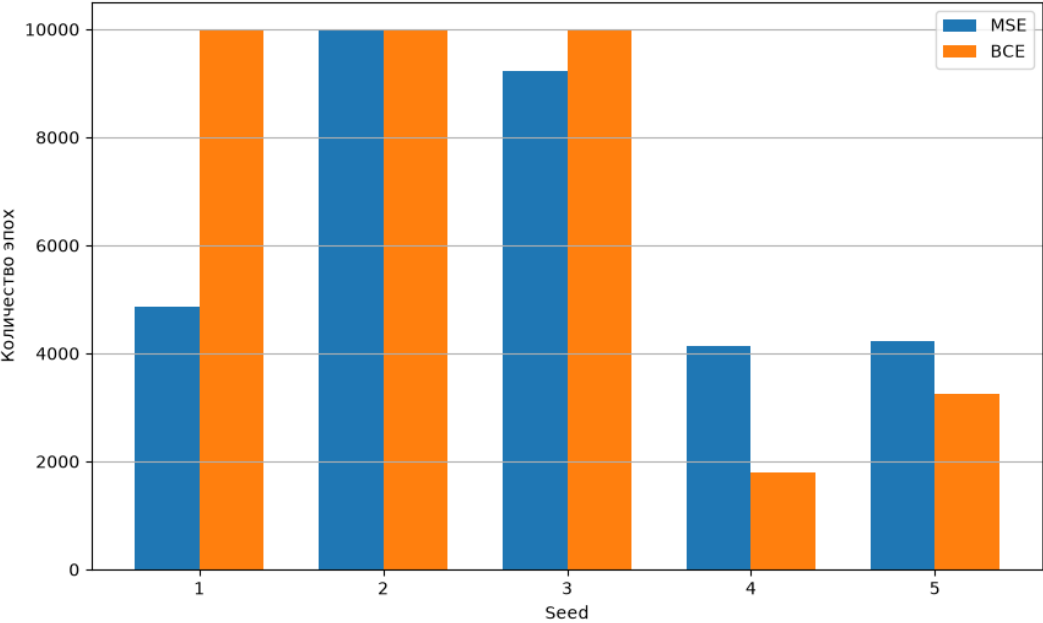
        if Es <= Ee or epoch >= max_epochs:
            break
    return W1, T1, W2, T2, errors
```

Результат

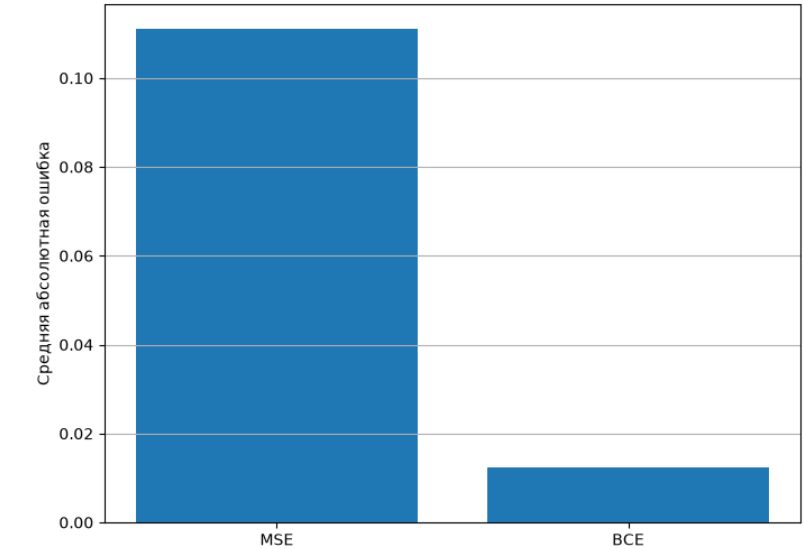
Сходимость MSE и Binary Cross-Entropy



Разброс числа эпох по 5 запускам

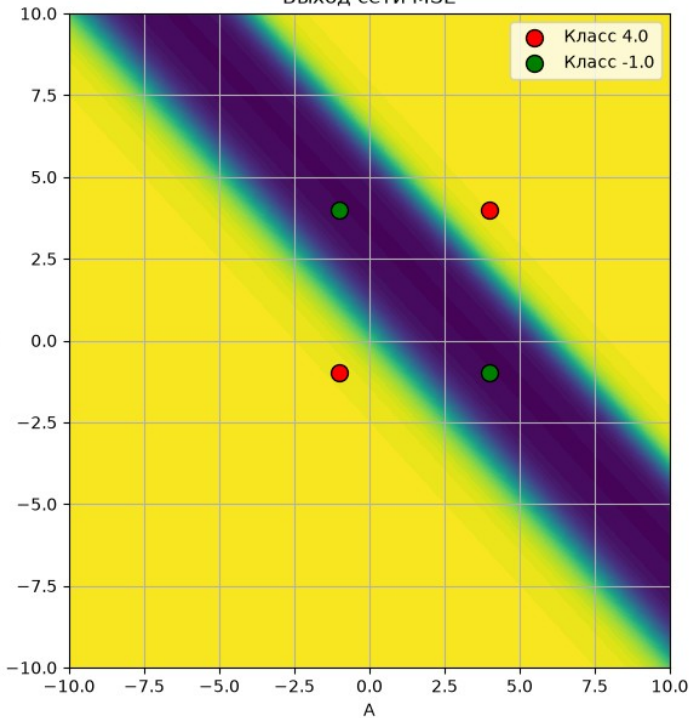


Сравнение точности восстановления исходной шкалы



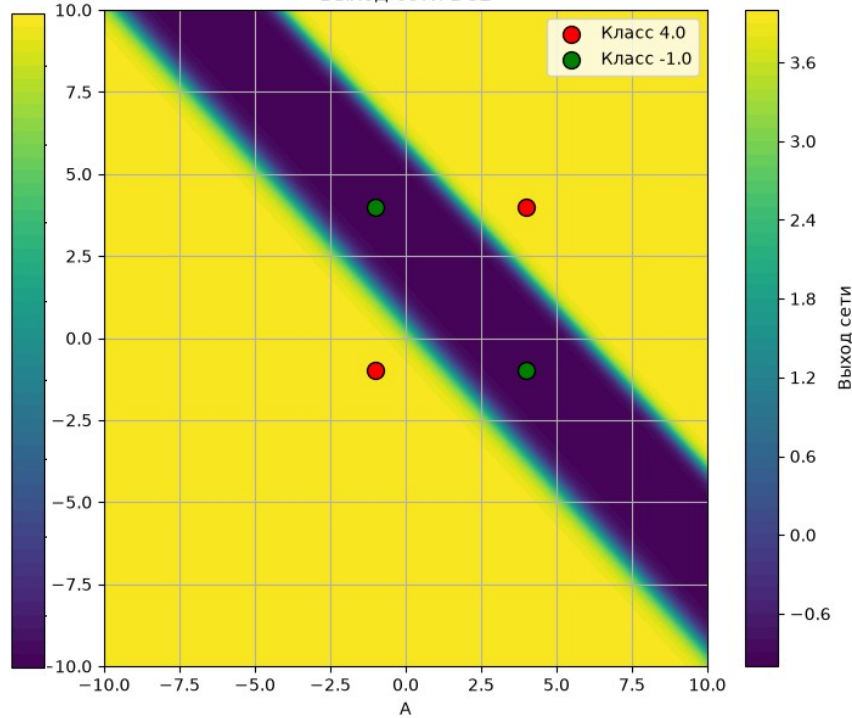
MSE

Выход сети MSE



BCE

Выход сети BCE



Исходные входы:

```
[[ 4.  4.]  
 [ 4. -1.]  
 [-1.  4.]  
 [-1. -1.]]
```

Нормализованные входы:

```
[[ -0.  -0.]  
 [ -0.   1.]  
 [  1.  -0.]  
 [  1.   1.]]
```

Исходные классы:

```
[ 4 -1 -1  4]
```

Нормализованные классы:

```
[-0.  1.  1. -0.]
```

КОНФИГУРАЦИЯ А — MSE

Эпох обучения: 4225

Финальная ошибка: 0.0009998327225944118

Ассурасу: 100.00%

Средняя абсолютная ошибка: 0.111063

Ответы сети:

Вход: [4. 4.], Ожидалось: 4.0000, Получено: 3.9009, Норм. выход: 0.0198

Вход: [4. -1.], Ожидалось: -1.0000, Получено: -0.8939, Норм. выход: 0.9788

Вход: [-1. 4.], Ожидалось: -1.0000, Получено: -0.8935, Норм. выход: 0.9787

Вход: [-1. -1.], Ожидалось: 4.0000, Получено: 3.8674, Норм. выход: 0.0265

КОНФИГУРАЦИЯ Б — BCE

Эпох обучения: 3974

Финальная ошибка: 0.009999524283952892

Ассурасу: 100.00%

Средняя абсолютная ошибка: 0.012483

Ответы сети:

Вход: [4. 4.], Ожидалось: 4.0000, Получено: 3.9898, Норм. выход: 0.0020

Вход: [4. -1.], Ожидалось: -1.0000, Получено: -0.9888, Норм. выход: 0.9978

Вход: [-1. 4.], Ожидалось: -1.0000, Получено: -0.9888, Норм. выход: 0.9978

Вход: [-1. -1.], Ожидалось: 4.0000, Получено: 3.9826, Норм. выход: 0.0035

5 запусков MSE

Seed=1: эпох=4866, ошибка=0.001000, accuracy=100.00%, MAE=0.110891, критерий=да

Seed=2: эпох=10000, ошибка=0.333998, accuracy=75.00%, MAE=1.695475, критерий=нет
Seed=3: эпох=9225, ошибка=0.001000, accuracy=100.00%, MAE=0.110432, критерий=да
Seed=4: эпох=4128, ошибка=0.001000, accuracy=100.00%, MAE=0.111081, критерий=да
Seed=5: эпох=4236, ошибка=0.001000, accuracy=100.00%, MAE=0.111051, критерий=да

5 запусков BCE

Seed=1: эпох=10000, ошибка=1.948081, accuracy=75.00%, MAE=1.763828, критерий=нет
Seed=2: эпох=10000, ошибка=1.948186, accuracy=75.00%, MAE=1.763886, критерий=нет
Seed=3: эпох=10000, ошибка=1.948145, accuracy=75.00%, MAE=1.763875, критерий=нет
Seed=4: эпох=1795, ошибка=0.009997, accuracy=100.00%, MAE=0.012480, критерий=да
Seed=5: эпох=3250, ошибка=0.009991, accuracy=100.00%, MAE=0.012472, критерий=да

Проверка 4 точек XOR

A=4, B=4 -> y=0.0020, y_real=3.9898, класс=4
A=4, B=-1 -> y=0.9978, y_real=-0.9888, класс=-1
A=-1, B=4 -> y=0.9978, y_real=-0.9888, класс=-1
A=-1, B=-1 -> y=0.0035, y_real=3.9826, класс=4

Введите пару чисел (A, B): 5 5
Нормализованный выход: 0.0015
Выход в исходной шкале: 3.9926
Ближайший класс: 4

Введите пару чисел (A, B): 5 0
Нормализованный выход: 0.9890
Выход в исходной шкале: -0.9449
Ближайший класс: -1

Введите пару чисел (A, B): 0 5
Нормализованный выход: 0.9874
Выход в исходной шкале: -0.9371
Ближайший класс: -1

Введите пару чисел (A, B): 0 0
Нормализованный выход: 0.1478
Выход в исходной шкале: 3.2612
Ближайший класс: 4

Вывод:

1. Какая из функций потерь (MSE или BCE) обеспечила более быструю и/или более устойчивую сходимость на задаче XOR с числовыми классами? Согласуется ли этот результат с известным теоретическим преимуществом BCE для задач классификации?

BCE в одном запуске сошлась немного быстрее: 3974 эпохи против 4225 у MSE. Однако по 5 запускам MSE оказалась устойчивее: критерий достигнут в 4/5 случаев, BCE - в 2/5. Результат частично согласуется с теоретическим преимуществом BCE для бинарной классификации, но устойчивость зависит также от инициализации и параметров обучения.

2. Как нормализация целевых значений (приведение c_0/c_1 к 0/1) повлияла на процесс обучения по сравнению с обучением напрямую на числах из диапазона $[-10; 10]$ (как в ЛР №1 с MSE)? Изменилась ли скорость и стабильность сходимости?

Нормализация перевела классы 4 и -1 в 0 и 1, что соответствует диапазону выхода сигмоиды и требованиям BCE. Без нормализации сеть бы не обучилась

3. Какова точность восстановления исходной числовой шкалы $[c_0; c_1]$ (средняя абсолютная ошибка) при использовании BCE с последующим пересчётом $\hat{y} \rightarrow y_{\text{real}}$, по сравнению с прямым обучением на MSE, где сеть сразу выдаёт

значение в нужной шкале? Есть ли здесь компромисс между удобством классификации (BCE) и точностью регрессии к исходной шкале?

Для BCE после обратного пересчёта $MAE = 0.012483$, для MSE - $MAE = 0.111063$. В данном эксперименте BCE обеспечила более точное восстановление исходной шкалы.

4. Насколько устойчивы обе конфигурации к случайной инициализации весов (по результатам серии из 5 запусков)? Наблюдались ли случаи недостижения критерия остановки или попадания в локальный минимум?

MSE достигла критерия в 4 из 5 запусков, BCE - в 2 из 5. В обоих случаях были неудачные запуски с точностью, что указывает на зависимость обучения от начальной инициализации и возможное попадание в локальные минимумы.

5. Подтвердилось ли на данном варианте (со своими c_0 и c_1) сделанное в ЛР №1 наблюдение о принципиальной линейной неразделимости задачи XOR независимо от того, какие конкретно числовые значения используются для кодирования классов 0 и 1?

Да, замена классов 0/1 на 4/-1 не изменила структуру XOR. Четыре точки по-прежнему линейно неразделимы одной прямой.

6. Как соотносится выбор функции потерь (MSE vs BCE) с общим выводом из предыдущей лабораторной о необходимости многослойной архитектуры для XOR – влияет ли тип функции потерь на этот вывод, или он определяется исключительно архитектурой сети?

Выбор MSE или BCE не меняет вывода о необходимости скрытого слоя. Линейная неразделимость XOR определяется архитектурой задачи, поэтому для её решения необходима нелинейная многослойная сеть.

7. Что показывает визуализация разделяющей поверхности на плоскости (A, B) для обеих конфигураций? Совпадает ли форма границы между классами с ожидаемой для XOR (пересечение двух полуплоскостей, формируемых скрытым слоем), и отличается ли она визуально между конфигурацией А (MSE) и конфигурацией Б (BCE)?

Поверхность показывает нелинейное разделение пространства (A, B) на области двух классов. Для обеих конфигураций сохраняется характерная структура XOR. При BCE выходы ближе к 0 и 1, поэтому разделение выражено более резко; при MSE переход более плавный.